

Actividad 1. Primera PKI: creación y despliegue básico de certificados X.509

Ana Trinidad Alonso Ansón

Grado en Ciberseguridad, Universidad Internacional de La Rioja (UNIR)

Seguridad en Comunicaciones Inalámbricas

Prof. Álvaro Vallega Moreno

9 de noviembre de 2026

Índice

Índice 2

Actividad 1. Primera PKI: creación y despliegue básico de certificados X.509 3

Introducción 3

1. Creación de la autoridad certificadora (CA) y emisión de certificados 3

1.1. Estructura de directorios y archivo openssl.cnf 3

1.2. Clave y certificado raíz 5

1.3. Certificado del servidor 6

1.4. Certificado de cliente (EAP-TLS) 7

1.5. Verificación de la cadena y registro de la CA 9

1.6. ¿Qué contiene un certificado X.509 y qué función tiene la CA? 9

2. Configuración de un servidor HTTPS local 10

2.1. Servidor con Python (http.server) 10

2.2. Comprobación de la conexión TLS 10

2.3. Importación de la CA en el navegador 11

2.4. Acceso a https://localhost:8443 sin advertencias 12

Conclusiones 14

Referencias 14

Actividad 1. Primera PKI: creación y despliegue básico de certificados X.509

Introducción

Este informe documenta la construcción de una pequeña infraestructura de clave pública (PKI) con OpenSSL: una autoridad certificadora (CA) raíz, un certificado de servidor para *localhost* y un certificado de cliente apto para autenticación EAP-TLS. Después se despliega un servidor HTTPS local con Python y se importa la CA en Firefox para comprobar que la conexión es de confianza. Todos los comandos se han ejecutado realmente y las figuras son capturas de pantalla del terminal (xterm) y del navegador Firefox.

Entorno. Linux x86-64 (equipo «kali», usuario anat) con OpenSSL 3.0.13, Python 3 y Firefox con la base NSS de su perfil (*~/.mozilla/firefox/kali.default-esr*). Los comandos son idénticos en Kali Linux. Los comandos principales se incluyen en texto junto a cada paso para facilitar su lectura y reproducción.

Contraseñas. Las contraseñas utilizadas (*unir1234*) son exclusivamente de laboratorio. En un entorno real se emplearían mecanismos seguros de introducción y custodia de secretos para evitar su exposición en el historial y en los argumentos de los procesos.

1. Creación de la autoridad certificadora (CA) y emisión de certificados

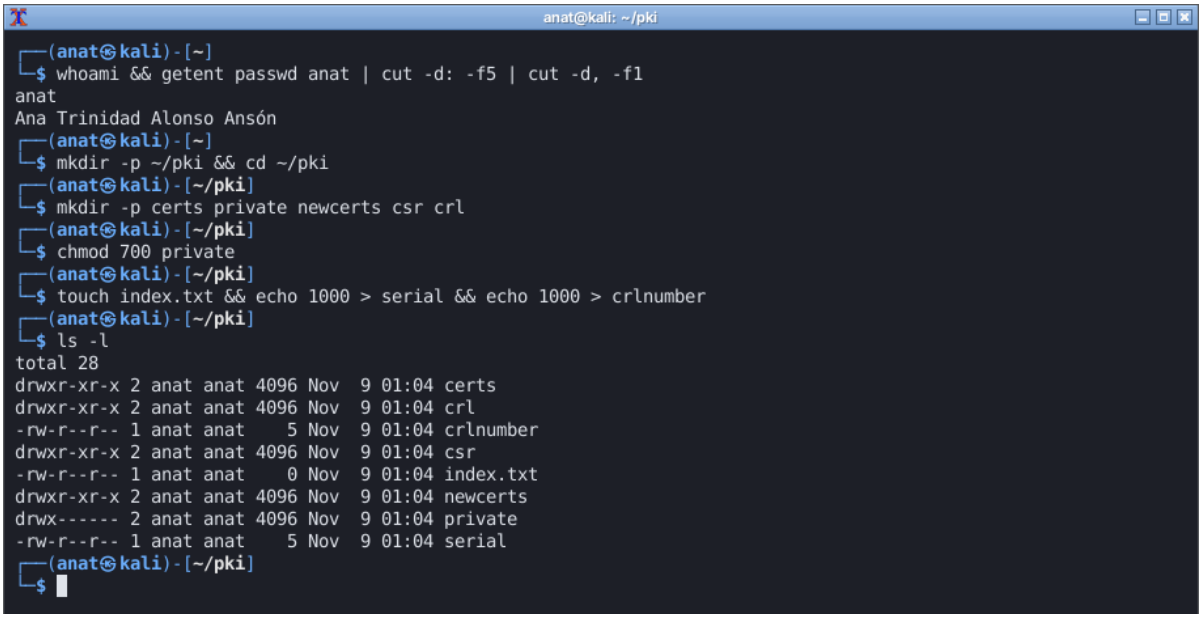
1.1. Estructura de directorios y archivo openssl.cnf

Se crea el árbol de trabajo clásico de una CA: *certs/* (certificados), *private/* (claves, con permisos 700), *newcerts/* (copias de lo emitido), *csr/* (peticiones) y *crl/*. Además se inicializan *index.txt* (base de datos de emisiones) y *serial* (siguiente número de serie).

```
mkdir -p ~/pki && cd ~/pki
mkdir -p certs private newcerts csr crl
chmod 700 private
touch index.txt && echo 1000 > serial && echo 1000 > crlnumber
nano openssl.cnf     # contenido en la Figura 2
```

Figura 1

Creación del entorno de trabajo de la CA

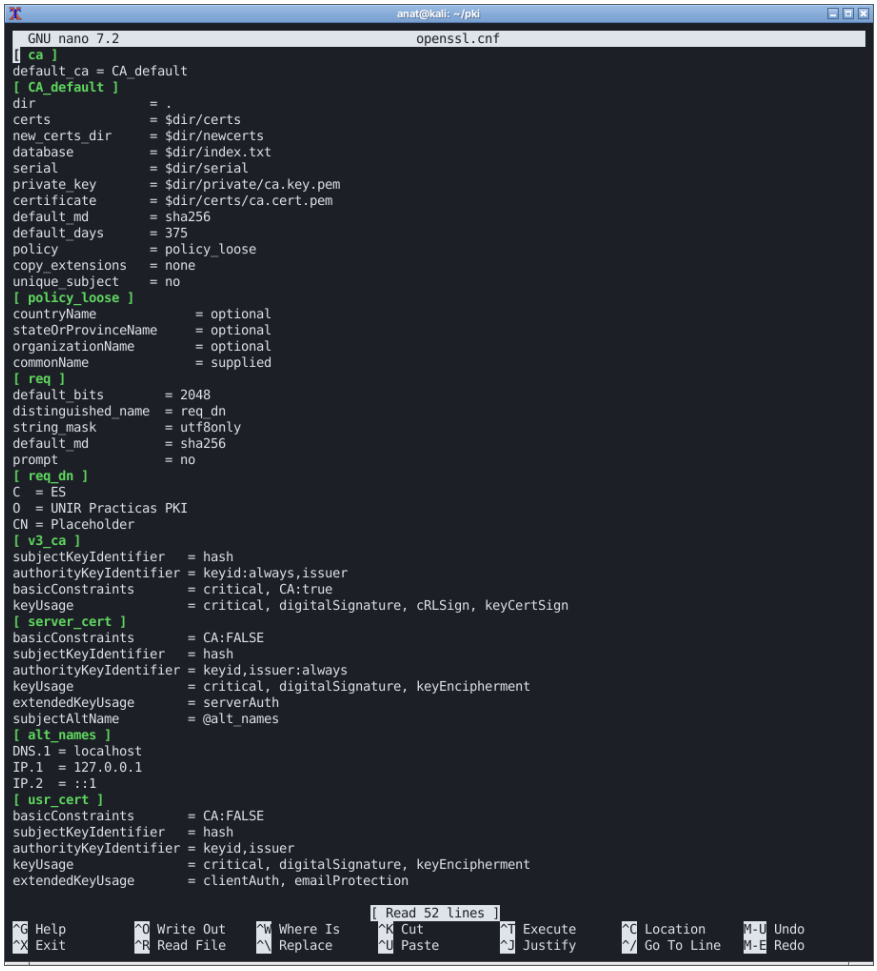


Nota. whoami identifica al usuario anat (Ana Trinidad Alonso Ansón), que crea el árbol de la CA en ~/pki.

El archivo *openssl.cnf* se adapta a este entorno: define la sección *[CA_default]* con las rutas anteriores, SHA-256 y política laxa, y tres perfiles de extensiones: *v3_ca* (CA:true, keyCertSign, cRLSign), *server_cert* (serverAuth y *subjectAltName* con localhost, 127.0.0.1 y ::1, imprescindible porque los navegadores modernos ignoran el CN) y *usr_cert* (clientAuth, necesario en EAP-TLS).

Figura 2

Archivo openssl.cnf editado con nano



Nota. Configuración de la CA, políticas y perfiles de extensiones para CA, servidor y cliente.

1.2. Clave y certificado raíz

Se genera una clave RSA de 4096 bits protegida con AES-256 y un certificado autofirmado válido 10 años con *openssl req -x509*. La clave privada queda en modo 400 para que solo pueda leerla su propietario.

```
openssl genrsa -aes256 -out private/ca.key.pem 4096
chmod 400 private/ca.key.pem
openssl req -config openssl.cnf -key private/ca.key.pem -new -x509 -days 3650 -sha256 \
  -extensions v3_ca -subj "/C=ES/O=UNIR Practicas PKI/CN=UNIR Practicas Root CA" \
  -out certs/ca.cert.pem
openssl x509 -noout -text -certopt no_pubkey,no_sigdump -in certs/ca.cert.pem
```

Figura 3

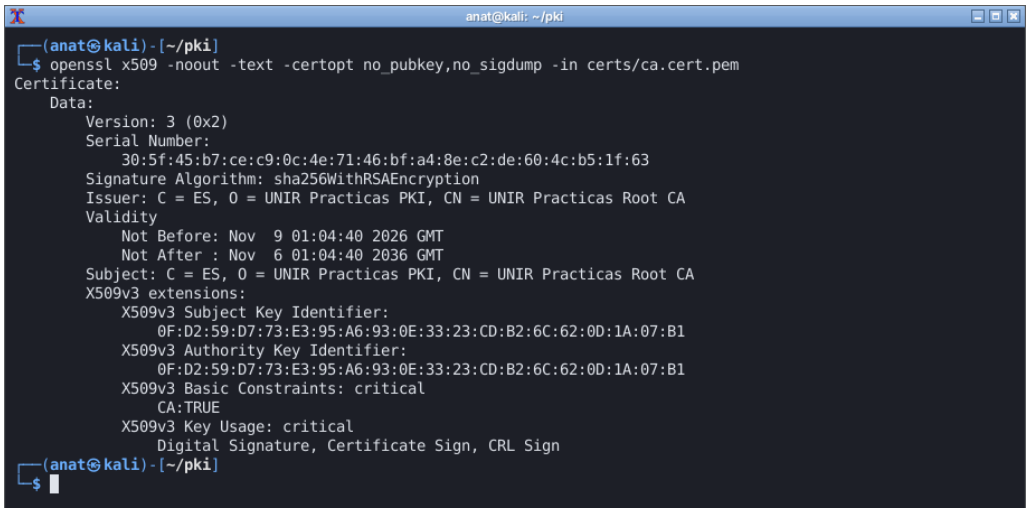
Generación de la clave privada y del certificado raíz



Nota. openssl no muestra salida cuando la operación tiene éxito; ls confirma los permisos 400 de la clave.

Figura 4

Contenido del certificado raíz (openssl x509 -text)



Nota. Se observan Issuer = Subject (autofirmado), CA:TRUE, Key Usage crítico y los identificadores SKI/AKI.

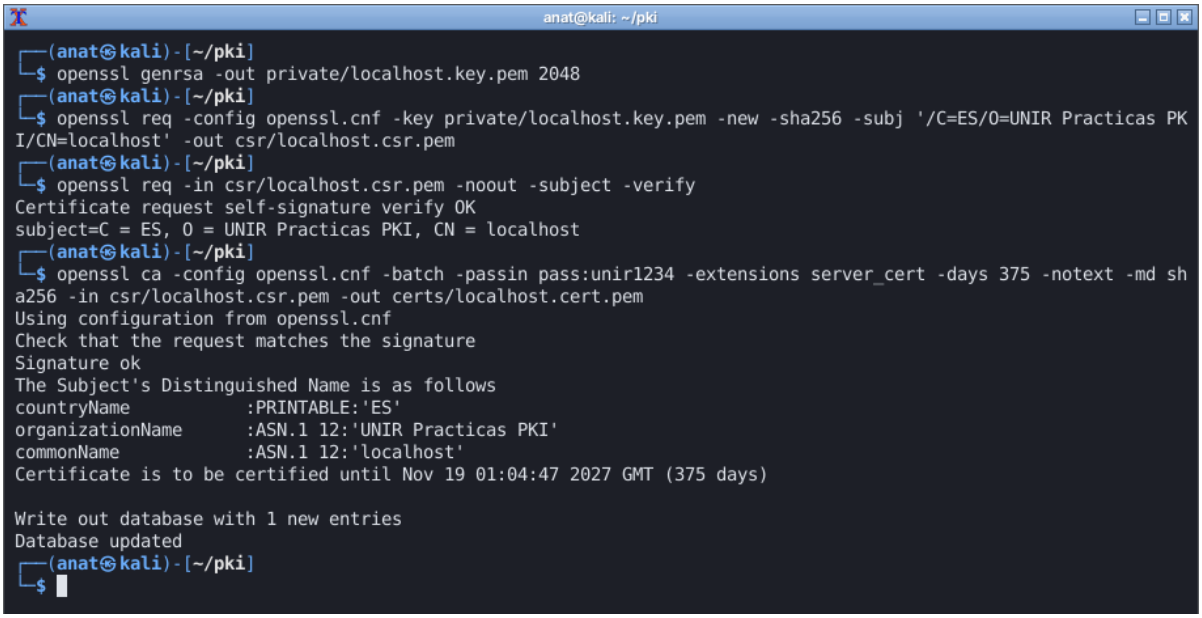
1.3. Certificado del servidor

Se crea una clave RSA de 2048 bits, una petición CSR con CN=localhost y se firma con *openssl ca* usando el perfil *server_cert*. La CA registra la emisión (número de serie 1000) en *index.txt*.

```
openssl genrsa -out private/localhost.key.pem 2048
openssl req -config openssl.cnf -key private/localhost.key.pem -new -sha256 \
  -subj "/C=ES/O=UNIR Practicas PKI/CN=localhost" -out csr/localhost.csr.pem
openssl req -in csr/localhost.csr.pem -noout -subject -verify
openssl ca -config openssl.cnf -extensions server_cert -days 375 -notext -md sha256 \
  -in csr/localhost.csr.pem -out certs/localhost.cert.pem
```

Figura 5

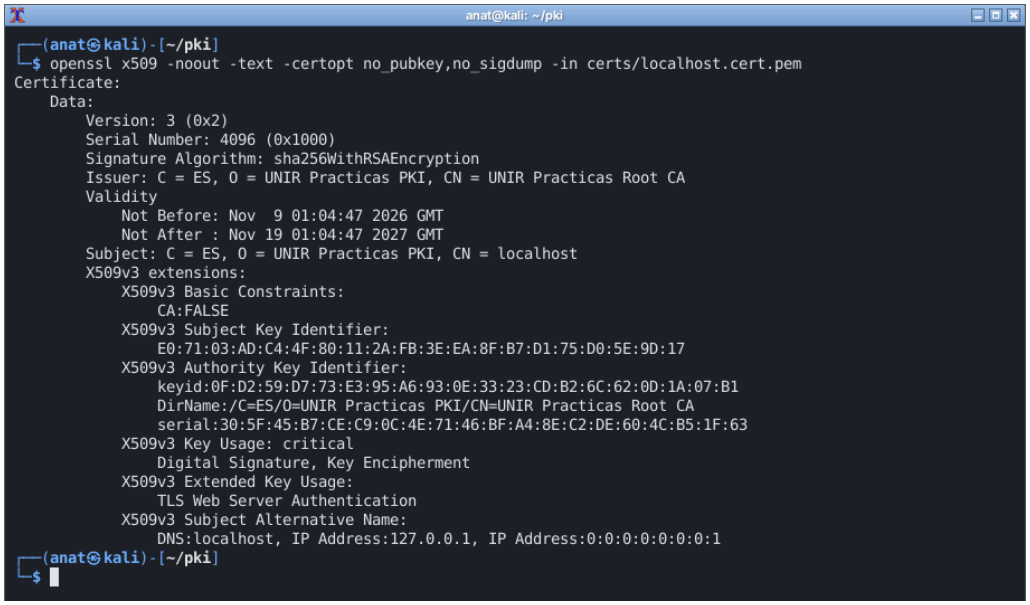
Clave, CSR y firma del certificado del servidor



Nota. La verificación de la CSR (verify OK) precede a la firma por la CA.

Figura 6

Detalle del certificado del servidor



Nota. Extensiones: CA:FALSE, serverAuth y SAN DNS:localhost, IP:127.0.0.1, IP:::1.

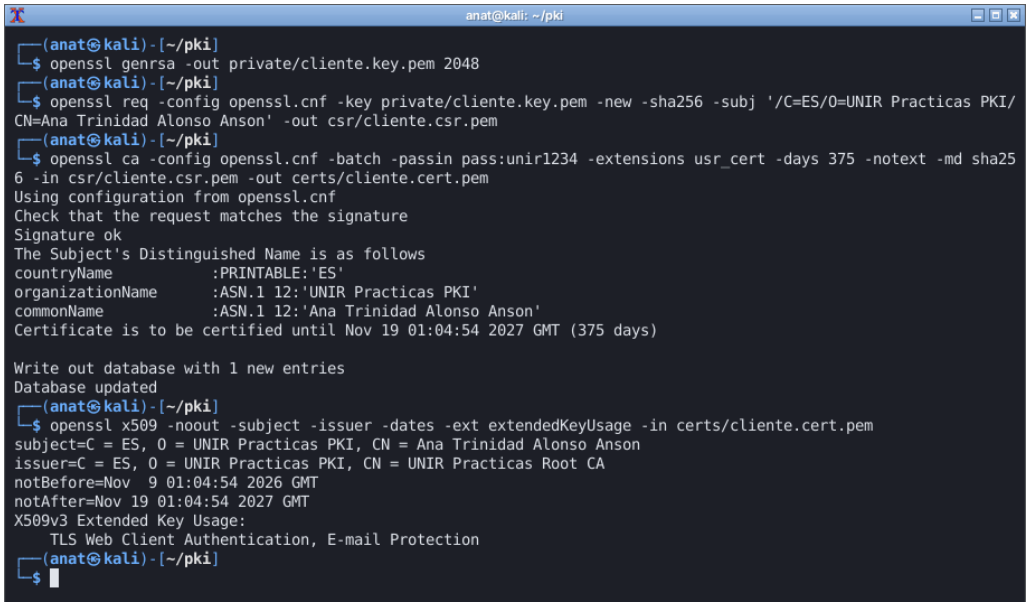
1.4. Certificado de cliente (EAP-TLS)

Para su uso en autenticación EAP-TLS se emite un certificado de cliente con el perfil *usr_cert* (extendedKeyUsage clientAuth). Se empaqueta junto con su clave y la CA en un archivo PKCS#12 (*cliente.p12*), que es el formato que importan navegadores, sistemas operativos y supplicants como wpa_supplicant.

```
openssl genrsa -out private/cliente.key.pem 2048
openssl req -config openssl.cnf -key private/cliente.key.pem -new -sha256 \
  -subj "/C=ES/O=UNIR Practicas PKI/CN=Ana Trinidad Alonso Anson" -out csr/cliente.csr.pem
openssl ca -config openssl.cnf -extensions usr_cert -days 375 -notext -md sha256 \
  -in csr/cliente.csr.pem -out certs/cliente.cert.pem
openssl pkcs12 -export -inkey private/cliente.key.pem -in certs/cliente.cert.pem \
  -certfile certs/ca.cert.pem -name "Cliente EAP-TLS" -out certs/cliente.p12
```

Figura 7

Emisión del certificado de cliente

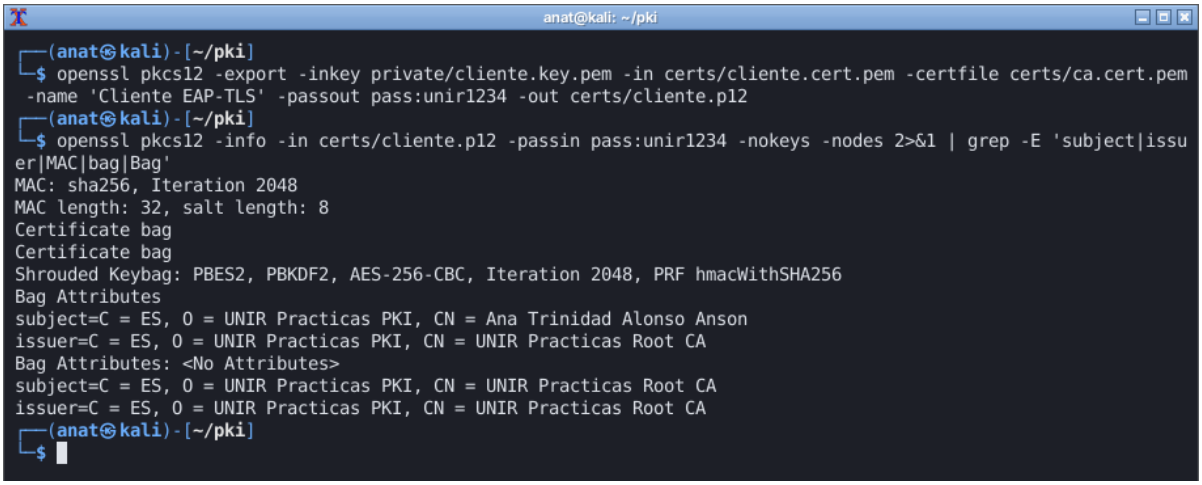


Nota. CN=Ana Trinidad Alonso Anson, emitido por UNIR Practicas Root CA (serie 1001).

El certificado generado incorpora la extensión clientAuth y está preparado para su utilización en un entorno de autenticación EAP-TLS. En esta práctica se demuestra su emisión y empaquetado, pero no se realiza una autenticación EAP-TLS contra un servidor RADIUS.

Figura 8

Empaquetado PKCS#12 del cliente



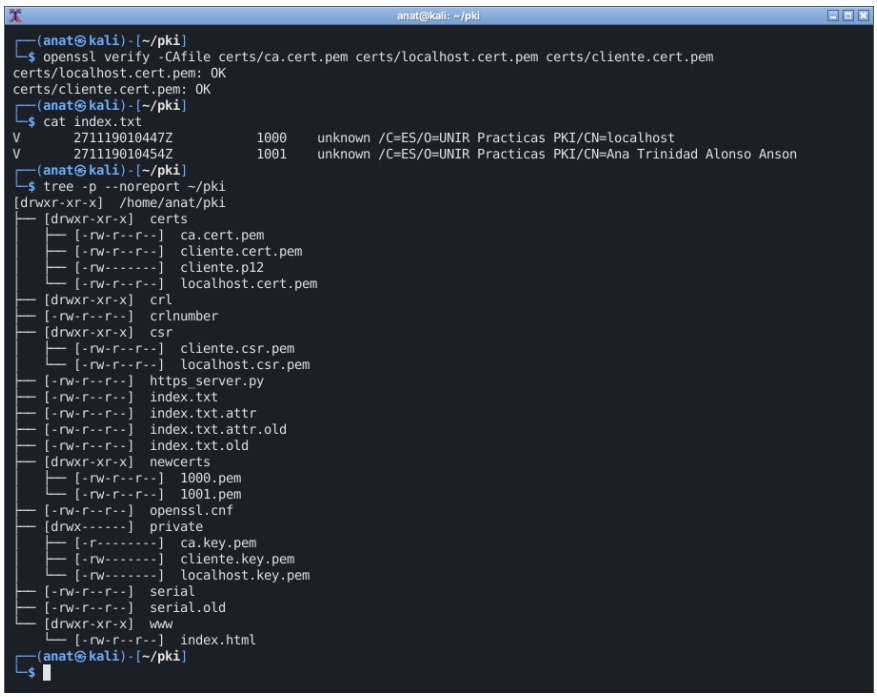
Nota. El .p12 contiene el certificado de cliente, la CA y la clave privada cifrada (shrouded keybag).

1.5. Verificación de la cadena y registro de la CA

```
openssl verify -CAfile certs/ca.cert.pem certs/localhost.cert.pem certs/cliente.cert.pem
cat index.txt
```

Figura 9

Verificación con openssl verify y estado de la CA



Nota. Ambos certificados validan contra la CA; index.txt muestra las dos emisiones con estado V (válido) y tree la estructura final con sus permisos.

1.6. ¿Qué contiene un certificado X.509 y qué función tiene la CA?

Un certificado X.509 (RFC 5280) liga una identidad a una clave pública mediante una firma digital. Incluye: versión (v3), número de serie, algoritmo de firma, emisor (*Issuer*), periodo de validez, sujeto (*Subject*), clave pública del sujeto y extensiones (*Basic Constraints*, *Key Usage*, *Extended Key Usage*, *Subject Alternative Name*, SKI/AKI), seguido de la firma del emisor (Housley et al., 2008).

La CA es el tercero de confianza: verifica la identidad del solicitante y firma su certificado con su clave privada. El cliente que confía en el certificado raíz puede verificar los certificados emitidos por la CA, siempre que se cumplan las condiciones de validación de la cadena, vigencia, uso previsto y demás comprobaciones aplicables. Por ello la clave privada de la CA se protege con contraseña y permisos estrictos. En una PKI real se añadiría una CA intermedia para mantener la raíz fuera de línea; aquí se emite directamente

desde la raíz, lo que el enunciado permite (la intermedia es opcional).

2. Configuración de un servidor HTTPS local

2.1. Servidor con Python (http.server)

Se usa *http.server* junto con el módulo *ssl*: se crea un contexto TLS de servidor, se cargan el certificado y la clave de *localhost* y se envuelve el socket. El servidor escucha en 127.0.0.1:8443, sirve *www/index.html* e indica al arrancar el usuario que lo ejecuta.

```
python3 https_server.py     # script de la Figura 10; escucha en 127.0.0.1:8443
```

Figura 10
Script https_server.py



Nota. Contexto SSLContext con load_cert_chain y socket envuelto.

Figura 11
Servidor HTTPS en ejecución



Nota. El servidor se ejecuta como usuario anat y registra tres peticiones a la página principal: dos respuestas 200 (contenido servido correctamente, desde curl) y una 304 (contenido no modificado, desde Firefox).

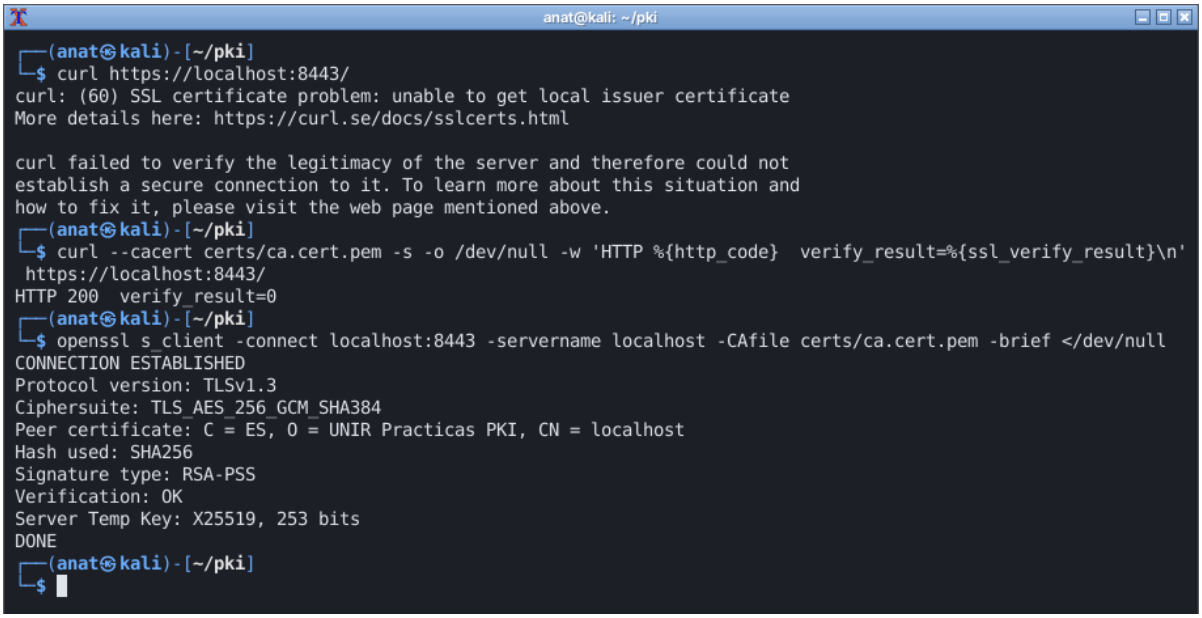
2.2. Comprobación de la conexión TLS

Primero se prueba sin confiar en la CA: curl falla con el error 60 (*unable to get local issuer certificate*), prueba de que el cliente no reconoce al emisor. Al indicar la CA con *--cacert* la verificación es correcta (código 0) y *openssl s_client* confirma TLS 1.3, AES-256-GCM y *Verification: OK*.

```
curl https://localhost:8443/                          # falla: CA no confiable
curl --cacert certs/ca.cert.pem https://localhost:8443/
openssl s_client -connect localhost:8443 -servername localhost -CAfile certs/ca.cert.pem -brief
```

Figura 12

Pruebas con curl y openssl s_client



Nota. Sin CA falla; con la CA el certificado se verifica correctamente.

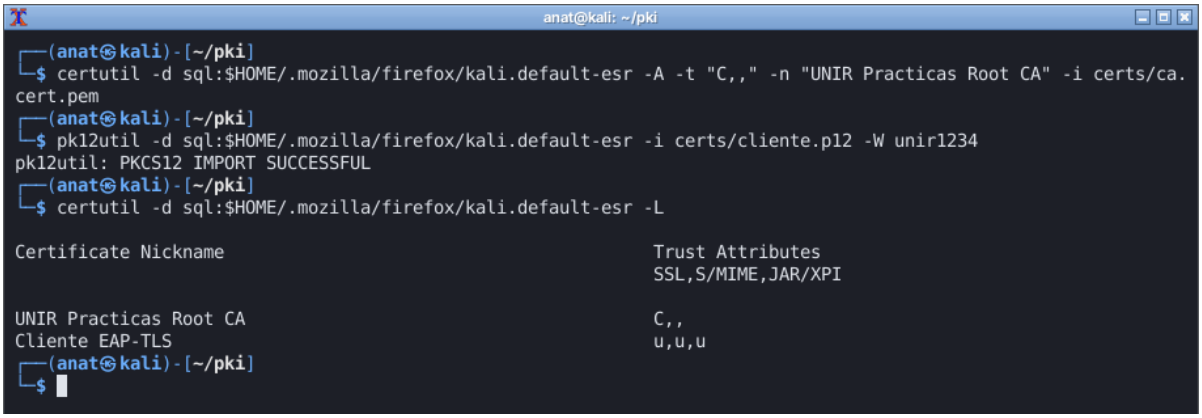
2.3. Importación de la CA en el navegador

Firefox guarda sus certificados en la base NSS de su perfil. Se importa la CA con *certutil -A -t "C,,"* (confianza para firmar certificados de servidor SSL) y el certificado de cliente con *pk12util*. En Firefox el equivalente gráfico es *Ajustes* > *Privacidad y seguridad* > *Ver certificados* > *Autoridades* > *Importar*, marcando «Confiar en esta CA para identificar sitios web».

```
PERFIL=~/.mozilla/firefox/<perfil>.default-esr      # con Firefox cerrado
certutil -d sql:$PERFIL -A -t "C,," -n "UNIR Practicas Root CA" -i certs/ca.cert.pem
pk12util -d sql:$PERFIL -i certs/cliente.p12
certutil -d sql:$PERFIL -L
```

Figura 13

Importación de la CA y del certificado de cliente en la base NSS



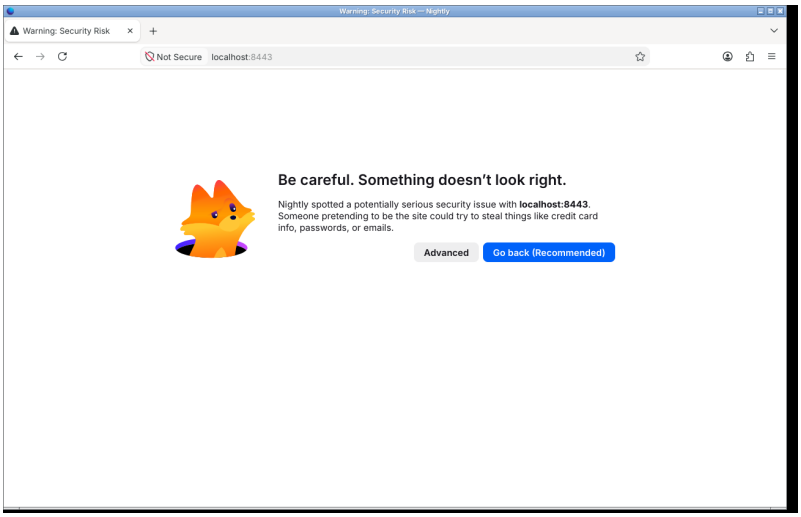
Nota. La CA queda con atributos C,, y el certificado de cliente con u,u,u (certificado personal con clave).

2.4. Acceso a https://localhost:8443 sin advertencias

Antes de la importación, Firefox bloquea el acceso con *SEC_ERROR_UNKNOWN_ISSUER*. Tras importar la CA, la página carga sin advertencias: el panel de seguridad indica que la conexión es segura y verificada por *UNIR Practicas PKI*, y el administrador de certificados de Firefox muestra el certificado de cliente importado.

Figura 14

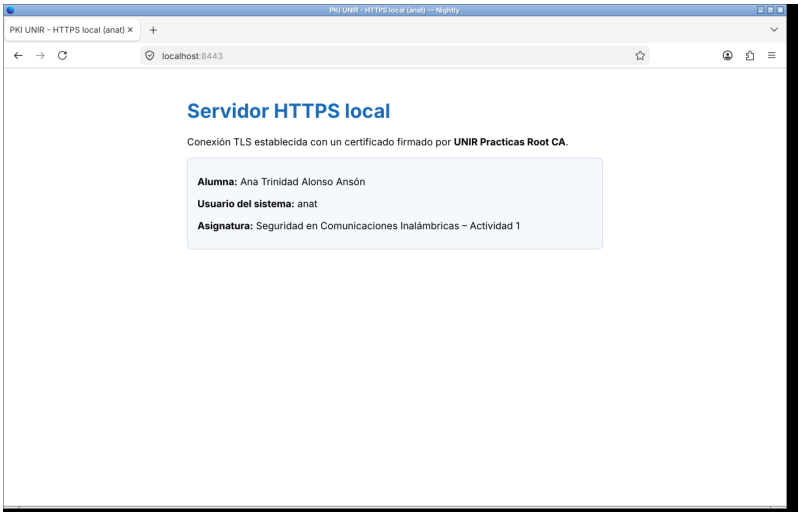
Firefox antes de importar la CA



Nota. Firefox bloquea el sitio con «Warning: Security Risk» (SEC_ERROR_UNKNOWN_ISSUER) porque no conoce al emisor.

Figura 15

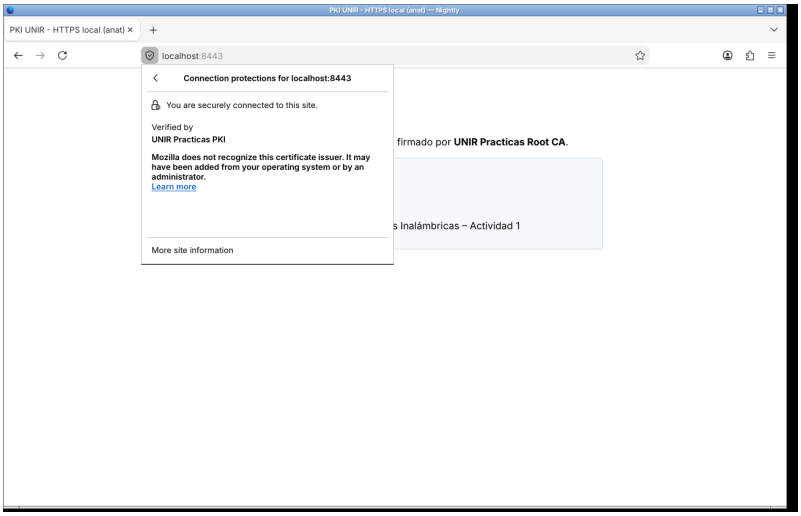
https://localhost:8443 en Firefox tras importar la CA



Nota. La página carga sin advertencias y muestra a la alumna y el usuario del sistema; el icono de la barra de direcciones ya no indica sitio no seguro.

Figura 16

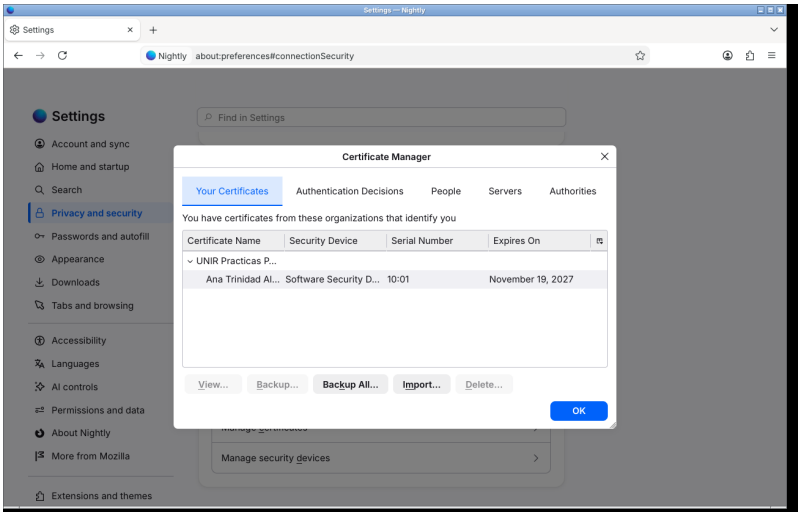
Detalle de la conexión en Firefox



Nota. «You are securely connected to this site. Verified by: UNIR Practicas PKI». El aviso adicional es el habitual de Firefox para autoridades importadas manualmente.

Figura 17

Administrador de certificados de Firefox: certificado de cliente



Nota. En «Your Certificates» aparece el certificado de cliente de Ana Trinidad Alonso Ansón, importado desde el .p12, con su caducidad.

Conclusiones

Se ha construido una PKI funcional: CA raíz, certificado de servidor con SAN y certificado de cliente con clientAuth, verificados con *openssl verify*. La confianza depende exclusivamente de que la CA esté instalada en el almacén del cliente: sin ella el navegador rechaza la conexión y con ella la acepta sin advertencias. Para producción convendría una CA intermedia, revocación (CRL/OCSP) y custodiar la clave raíz fuera de línea.

Referencias

Housley, R., Polk, W., Ford, W. y Solo, D. (2008). *Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile* (RFC 5280). IETF. https://www.rfc-editor.org/rfc/rfc5280

OpenSSL Project. (2024). *openssl-ca: sample minimal CA application* (versión 3.0). https://docs.openssl.org/3.0/man1/openssl-ca/

OpenSSL Project. (2024). *openssl-req: PKCS#10 certificate request and certificate generating command* (versión 3.0). https://docs.openssl.org/3.0/man1/openssl-req/

Python Software Foundation. (s. f.). *ssl — TLS/SSL wrapper for socket objects*. Python 3 documentation. https://docs.python.org/3/library/ssl.html

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). IETF. https://www.rfc-editor.org/rfc/rfc8446

Simon, D., Aboba, B. y Hurst, R. (2008). *The EAP-TLS authentication protocol* (RFC 5216). IETF. https://www.rfc-editor.org/rfc/rfc5216